A Comparative Analysis of Open-Source LLMs for Converting Architecture, Engineering and Construction Standards to the RASE Format

Eike Natan Sousa Brito[a,*], André Carvalho[a] and Marco Antônio Brasiel Sampaio[a]

[a]*Universidade Federal de Sergipe, Cidade Universitária Prof. José Aloísio de Campos, São Cristóvão, 49100-000, Sergipe, Brazil*

ARTICLE INFO

Keywords:
Machine Learning
Natural Language Processing
LLM
Prompt Engineering
BIM
RASE

ABSTRACT

The *Building Information Modeling* (BIM) methodology organizes the planning and execution of construction projects around a detailed three-dimensional model, reducing rework and anticipating interferences before construction begins. This model, however, does not by itself solve the verification of design compliance against regulatory technical standards, whose interpretive nature makes automation an open problem. The RASE methodology (*Requirement, Applicability, Selection, Exception*) addresses this gap by rewriting each normative excerpt as computable data, allowing BIM *software* to verify compliance automatically; converting standards into this format, however, is still done manually, rule by rule, which becomes a bottleneck for automation. Natural Language Processing, and Large Language Models (LLMs) in particular, emerges as a promising path to automate this conversion, and can be adapted to the task through Prompt Engineering, a lightweight technique that adjusts only the instruction sent to the model. This dissertation comparatively evaluates six open-source LLMs (Llama, Dolphin, Gemma, Mistral, Alpaca, and Qwen) on the automatic conversion of Brazilian AEC technical standards to the RASE format, using Prompt Engineering exclusively across three successive normalization levels (N1: atomic segmentation of normative sentences; N2: RASE operator identification; N3: semantic JSON structuring). The comparison was carried out over five experiments (EN1, EN2, EN1N2, EN3, and EN1N2N3) on a dataset of 79 annotated NBR 9050 standards, with thirteen metrics across five families (lexical, semantic, distance-based, generation-oriented, and classification), with the models served locally through Ollama under an identical dataset, prompt, and hardware protocol. The results show that the three-level decomposition, relying solely on Prompt Engineering, is sufficient to produce valid RASE representations, reaching semantic similarity close to 0.90 in the best scenarios; Llama achieved the best overall average (0.730), leading the N3-level stages (EN3 and EN1N2N3), while Dolphin offered the best cost-benefit ratio, around ninety times faster in the chained pipeline.

1. Introduction

Designing and reviewing civil engineering projects is a knowledge-intensive activity: it requires engineers to interpret regulations, reconcile heterogeneous information, and make decisions grounded in expertise accumulated over years of practice. Engineering Informatics studies how computational methods can support precisely these activities, turning the knowledge embedded in documents, models, and standards into a machine-processable form. Within this field, automating the verification of building designs against regulatory technical standards is a long-standing and still open problem, since these standards are written in natural language and demand interpretation. The present work can be characterized as a problem in this area: it uses Artificial Intelligence (AI) to convert technical standards into a computable representation that Building Information Modeling (BIM) tools can check automatically, a task that sits at the intersection of knowledge representation, Natural Language Processing, and engineering practice.

In Brazilian civil engineering, designing and inspecting works still involves a large amount of rework. Part of this waste stems from how projects are represented: 2D drawings, the sector's historical standard, hide clashes and omit information that only surfaces on the construction site. The move toward three-dimensional models and, more specifically, toward the *Building Information Modeling* (BIM) methodology emerged as a response to this problem (Eastman, Teicholz and Sacks, 2011).

*Corresponding author
eike.sousa@hotmail.com (E.N.S. Brito); andre@dcomp.ufs.br (A. Carvalho); marco.sampaio@ufs.br (M.A.B. Sampaio)
ORCID(s):

BIM organizes project planning and execution around a detailed 3D model. The methodology spans dimensions ranging from 3D to 7D: 3D for the geometric model itself; 4D for time scheduling; 5D for costs; 6D for sustainability; and 7D for post-construction operation management (Eastman et al., 2011). BIM is not a piece of *software* but a concept supported by computer systems such as CAD; it is implemented by tools such as Autodesk Revit and Graphisoft ArchiCAD.

Modeling in BIM makes it possible to visualize interferences before construction begins, reducing on-site rework. Typical applications range from design analysis to budgeting, including planning, modular coordination, and life-cycle management (Eastman et al., 2011).

BIM, however, does not by itself solve the verification of designs against regulatory standards. Regulatory (or technical) standards are documents that establish mandatory requirements, criteria, and procedures to ensure the safety, quality, and performance of buildings; in Brazil, they are issued by the Brazilian Association of Technical Standards (ABNT) as NBRs, such as NBR 9050 on accessibility. Each jurisdiction maintains its own set of rules, generally written in natural language and interpretive in nature, which makes automated compliance checking an open problem (Dimyadi and Amor, 2013; Solihin and Eastman, 2015).

Making these standards machine-checkable requires converting them into a computable representation. To this end, Hjelseth and Nisbet (2011) proposed the RASE methodology, which rewrites each normative excerpt as structured data decomposed into four attributes: **R**equirement (what must be met), **A**pplicability (where the rule applies), **S**election (options for complying with it), and **E**xception (cases in which the rule does not hold). Once organized in RASE, a standard enables tools integrated with BIM *software*, such as Revit, to validate a design element by element (Santos and Sampaio, 2021). The bottleneck lies upstream: converting a standard to RASE is still done manually, rule by rule, and because a BIM model may gather hundreds or thousands of elements subject to standards, the cost of this work grows quickly.

Automating this conversion requires recognizing patterns in natural-language text, a task for which Machine Learning (ML) is promising. The motivation is reinforced by BIM itself: the wider its adoption, the larger the volume of data generated by each project, attracting data scientists, professionals whose role is to turn large volumes of information into value for organizations (Davenport and Patil, 2012). ML, a subfield of AI, studies how to build algorithms that learn patterns directly from data, without explicitly programming the rules (Tan, Steinbach and Kumar, 2006); from the learned patterns, these models make predictions and respond to novel situations. In civil engineering, AI already automates steps that once relied on human inspection, from detecting structural pathologies to forecasting construction schedules (Rane, Choudhary and Rane, 2023; Liu, Zhang and Li, 2022).

Three recent examples illustrate this movement. Liu et al. (2022) combined BIM, ML, and design rules to optimize the construction assembly process, showing gains in efficiency and cost. Alawadhi and Yan (2023) trained Artificial Neural Networks on BIM data to recognize construction objects in photographs: synthetic images generated from the model produced effective semantic segmentation on real photographs, paving the way for the use of synthetic data in architectural problems. On another front, Saka, Taiwo, Saka, Oluleye, Dauda and Akanbi (2024) fed a model with 2,280 demolition projects to predict waste circularity: the system estimates the amounts of recyclable material and routes waste to the appropriate landfill, supporting planning.

When the object of analysis is the normative text itself, automation falls to Natural Language Processing (NLP), a subfield of AI devoted to the computational treatment of human language in tasks such as translation, summarization, and text classification. Within NLP, Generative AI gathers the models capable of producing new content (text, image, or audio) from what they learned during training. Among its most relevant manifestations are Large Language Models (LLMs) (Zhao, Zhou, Li, Tang, Wang, Hou, Min, Zhang, Zhang, Dong, Du, Yang, Chen, Chen, Jiang, Ren, Li, Tang, Liu, Nie and Wen, 2023), a category that includes OpenAI's ChatGPT (OpenAI, 2024) and Meta's Llama 3 (AI@Meta, 2024).

LLMs are AI systems aimed at understanding and producing human language at scale. Training involves massive volumes of text, allowing the model to capture linguistic and contextual nuances. Almost all current LLMs use some variation of the *transformer* architecture, chosen for its efficiency in processing long sequences (Zhao et al., 2023).

Adapting these models to specific domains can follow different paths. The lightest and most accessible of them is Prompt Engineering (PE): instead of retraining weights or building a retrieval infrastructure, one adjusts only the instruction sent to the model.

In the AEC sector, LLMs can assist both in reading standards and in converting them into machine-processable formats (Fuchs, Witbrock, Dimyadi and Amor, 2024). Yet the literature still has gaps: one of them is precisely the

interpretation of laws, codes, and regulatory standards, whose semantic complexity has historically challenged earlier AI methods (Fuchs et al., 2024).

Faced with this gap, this work decomposes the conversion of standards to RASE into three successive normalization levels, supported solely by Prompt Engineering: level **N1** segments the normative sentence into atomic rules; **N2** identifies, in each rule, the RASE operators; and **N3** structures the result as JSON. Over this decomposition, six open-source LLMs (Llama, Dolphin, Gemma, Mistral, Alpaca, and Qwen) are compared, served locally, across five experiments that evaluate each level in isolation and in a chained pipeline. Output quality is assessed by nine textual-similarity metrics across four families: lexical (FuzzyWuzzy and TF-IDF), semantic (SBERT, BERTimbau, and Multilingual), distance-based (*Word Mover's Distance* with FastText and NILC), and generation-oriented (BERTScore and ROUGE-L); these are complemented by four classification metrics (Accuracy, Precision, Recall, and F1-score).

The contributions of this work lie at the intersection of Architecture, Engineering and Construction (AEC) and Computing: for AEC, it applies NLP models to building-code checking and proposes successive normalization levels over the RASE methodology of Hjelseth and Nisbet (2011); for Computing, it brings LLMs well established in the literature to a new domain and subjects them to a systematic performance evaluation.

2. Background

2.1. Engineering Standards and the RASE Methodology

Technical standards form the regulatory framework of the AEC sector: they define the minimum requirements of safety, performance, quality, and interoperability for designs. At the international level, the International Organization for Standardization (ISO) publishes standards adopted across many countries; in BIM, the consolidated reference is the ISO 19650 series, devoted to organizing and digitizing information about construction works (ISO19650). Regardless of jurisdiction, each standard combines three textual layers: (i) definitions and references, with no direct verification value; (ii) prescriptive requirements, with numerical and geometric limits amenable to automated checking; and (iii) recommendations and exceptions, in free text, often conditioned on specific building categories. In all three layers the raw text is long, recursive, and heavily dependent on prior definitions, which makes the translation into computable rules a non-trivial and, today, predominantly manual task.

Applying these standards remains largely manual: the professional reads long passages, identifies the relevant requirements, and checks them item by item against the design. In a BIM model with hundreds or thousands of elements subject to simultaneous standards, this effort grows combinatorially; Eastman, Lee, Jeong and Lee (2009) show that manual checking is costly, fragile, and hardly scalable, motivating research into automated rule-based checking. Automating it, however, faces three obstacles: the ambiguity and cross-references of the natural language in which standards are written (Dimyadi and Amor, 2013); regional variation, which limits the reuse of what has already been formalized elsewhere (Dimyadi and Amor, 2013; Solihin and Eastman, 2015); and the semantic difficulty of distinguishing, within a passage, what is a requirement, an applicability, and an exception, which calls for a standardized, tool-interoperable schema (Solihin and Eastman, 2015).

The response the literature has been consolidating is to structure the standard into a computable schema before verification. Beach, Rezgui, Li and Kasim (2015) demonstrate that a rule-based system automates regulatory compliance checking in construction once standards are formalized. Along these lines, Hjelseth (2015) proposed two complementary methodologies: Tx3 and RASE. Tx3 classifies each piece of regulatory information as **Transcribe** (instructions directly transcribable into computable rules), **Transform** (instructions that must be rewritten while preserving meaning), and **Transfer** (imprecise requirements that demand expert analysis). Transcribable and transformable elements are rewritten and submitted to RASE, which marks each normative passage with four logical operators (Hjelseth and Nisbet, 2011): **R**equirement (the associated duty, usually expressed by the imperative "must"), **A**pplicability (to whom or to what the requirement applies), **S**election (a subset of the applicability), and **E**xception (the cases excluded from the rule). Each operator further receives structured attributes such as object, property, comparator, value, and unit, bringing the text closer to a directly verifiable form. For instance, in the sentence "the cross slope must be at most 2% for indoor floors", the applicability is "floors", the selection "indoor", and the requirement "cross slope less than or equal to 2%". Once organized in RASE, a standard enables tools integrated with BIM *software*, such as Revit, to validate a design element by element (Santos and Sampaio, 2021). Applying RASE manually, however, reintroduces the very bottleneck it aims to remove: marking operator by operator requires an expert and does not scale to the volume of standards in the sector. At its core, identifying Requirement, Applicability, Selection, and Exception

in a sentence is a text classification and structuring problem, exactly the kind of task that Machine Learning techniques learn from examples.

2.2. Large Language Models and Textual Validation Metrics

Large Language Models (LLMs) are AI models that combine Machine Learning, deep neural networks, and Natural Language Processing techniques to understand and generate human language (Touvron, Lavril, Izacard, Martinet, Lachaux, Lacroix, Rozière, Goyal, Hambro, Azhar, Rodriguez, Joulin, Grave and Lample, 2023). They are typically pre-trained in an unsupervised fashion over billions of words, learning to predict sentence structure probabilistically, and adopt the *Transformer* architecture (Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser and Polosukhin, 2017), efficient at handling contextual dependencies over long sequences. The release of ChatGPT and GPT-4 (OpenAI, 2024) popularized these models, and, as a response to closed models, open-weight alternatives such as LLaMA emerged (Touvron et al., 2023; AI@Meta, 2024). Among the techniques that specialize an LLM for a domain task without changing its weights, Prompt Engineering is the lightest: it consists of carefully designing the instruction sent to the model, an essential practice for obtaining context-appropriate responses (Brown, Mann, Ryder, Subbiah et al., 2020).

Evaluating an LLM's output in text-conversion tasks requires comparing it to a reference. Since exact matching is rarely desirable, because a correct paraphrase must also count as a hit, a set of similarity metrics is used, organized into four families (Jurafsky and Martin, 2009). The **lexical** ones compare the surface form of the strings: FuzzyWuzzy, based on the Levenshtein distance, and cosine similarity over TF-IDF vectors (Manning, Raghavan and Schütze, 2008). The **semantic** ones rely on sentence embeddings that bring texts of equivalent meaning closer, such as Sentence-BERT (Reimers and Gurevych, 2019), BERTimbau (specialized in Portuguese) (Souza, Nogueira and Lotufo, 2020), and multilingual models. The **distance-based** ones measure the cost of aligning the word embeddings of one text to those of another, as in the Word Mover's Distance (Kusner, Sun, Kolkin and Weinberger, 2015), runnable over FastText (Bojanowski, Grave, Joulin and Mikolov, 2017) or NILC embeddings. The **generation-oriented** ones, originating in machine translation and summarization, measure how much the output reproduces the reference: BERTScore (Zhang, Kishore, Wu, Weinberger and Artzi, 2020), which aligns tokens by contextual similarity, and ROUGE-L (Lin, 2004), based on the longest common subsequence. To these we add classification metrics (accuracy, precision, recall, and F1-score), derived from the confusion matrix, which make omissions and hallucinations explicit by treating evaluation as an information-extraction problem (Manning et al., 2008).

2.3. Related Work

The integration of BIM with generative AI technologies is still little explored in the AEC sector, but it is the subject of a recent wave of work. Rane et al. (2023) analyze the integration between BIM and models such as ChatGPT and Bard, identifying gains in design and decision-making, but pointing to persistent obstacles (lack of data standardization, security shortcomings, and the models' difficulty in interpreting the semantics of BIM data) and proposing an adoption framework. Lin (2023) present JULE, a chatbot that combines BIM and NLP to deliver real-time, on-site queries about component dimensions without opening the conventional interfaces, with high accuracy in intent recognition but conditioned on well-structured BIM data. Sammour, Xu, Wang, Hu and Zhang (2024) apply GPT-3.5 and GPT-4 to 385 safety questions and three Prompt Engineering techniques (Prompt Direct, Chain-of-Thought, and Few-Shot): GPT-4 reached 84.6% accuracy against 73.8% for GPT-3.5, but both failed on emergency and calculation questions, exposing the fragility of technical reliability. Finally, Samsami (2024) show that generic prompts do not work in the AEC sector and that specific prompts reduce errors, although hallucinations in complex tasks and the need for external validation persist. Taken together, these works confirm the potential of LLMs in the sector but reveal a common gap: the absence of a structured, formal representation that translates normative content into computable rules. Without it, LLMs extract text and generate insights but do not convert the standard into something directly machine-verifiable, exactly the point this work addresses by coupling LLMs to the RASE methodology.

3. RASE-PE: Converting Standards to RASE with Prompt Engineering

The proposed methodology, here named **RASE-PE**, rests on three decisions: (i) decomposing the conversion of standards into the RASE format across three successive normalization levels (N1, N2, and N3); (ii) using Prompt Engineering as the sole model-specialization technique, without changing weights or building a retrieval infrastructure; and (iii) systematically comparing six open-source LLMs served locally through Ollama, under an identical dataset, prompt, and hardware protocol. The following subsections detail the three-level decomposition, the

Prompt Engineering employed, the evaluated models, the dataset, the validation metrics, and, finally, the computational environment and the code structure.

3.1. Three-Level Normalization

Moving from the raw text of a standard to the RASE representation in JSON requires three distinct reasoning steps: syntactic segmentation, semantic classification, and structured-attribute extraction. Asking for all of this in a single LLM call tends to degrade the output, with hallucinations, omissions, and invalid JSON. RASE-PE decomposes the task into three successive levels, isolating each decision and allowing each step to be validated separately:

- **N1: Atomic Segmentation.** Normative sentences often embed more than one coordinated rule (for example, "the cross slope must be at most 2% for indoor floors and at most 3% for outdoor floors" contains two rules). N1 breaks the original sentence into atomic items, with a single rule per item, a precondition for the next steps to correctly assign the RASE operators.
- **N2: RASE Operator Identification.** From each atomic rule, N2 identifies which spans correspond to Requirement, Applicability, Selection, and Exception. It is the core of the formalization: without it, the text remains in natural language.
- **N3: Semantic JSON Structuring.** Classifying a span as R, A, S, or E still leaves it in natural language; for automated checking against a BIM model, the data must be extracted into structured fields. N3 produces a JSON with the fields *type*, *object*, *property*, *comparation*, *target*, and *unit*.

The chaining N1 → N2 → N3 forms the complete RASE-PE *pipeline* (Figure 1), from raw text to the structured RASE representation. To isolate the contribution of each level and measure error propagation along the chain, the evaluation is organized into five experiments, presented in Section 4.

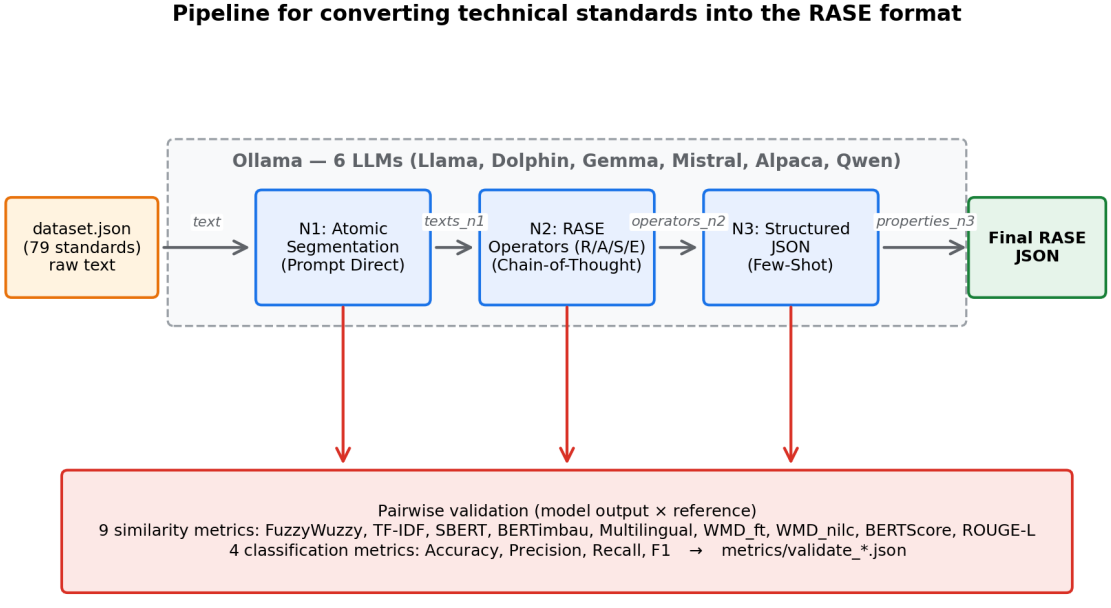

Figure 1: RASE-PE *pipeline* for converting technical standards into the RASE format.

3.2. Prompt Engineering

The specialization of the LLMs was carried out exclusively through Prompt Engineering: all task adaptation lives in the prompts, organized by level and by operator. Three classic techniques were applied, each chosen for the nature of the level it acts on. **N1** uses *Prompt Direct*, a direct and objective instruction suited to a structurally simple task

Table 1
Evaluated open-source LLMs, with origin and the tag served through Ollama.

Model	Origin	Tag served through Ollama	Parameters
Llama	Meta	`llama3.1:8b`	8B
Dolphin	cnmoro (LLaMA-3 PT)	`cnmoro/llama-3-8b-dolphin-portuguese-v0.3:4_k_m`	8B (Q4_K_M)
Gemma	brunoconterato (PT-BR)	`brunoconterato/Gemma-3-Gaia-PT-BR-4b-it:f16`	4B (FP16)
Mistral	cnmoro (PT)	`cnmoro/mistral_7b_portuguese:q4_K_M`	7B (Q4_K_M)
Alpaca	splitpierre (Bode PT-BR)	`splitpierre/bode-alpaca-pt-br:13b-Q4_0`	13B (Q4_0)
Qwen	cnmoro (PT)	`cnmoro/Qwen2.5-0.5B-Portuguese-v1:q4_k_m`	0.5B (Q4_K_M)

(segmenting the sentence into sub-rules) and economical in tokens and latency. **N2** uses *Chain-of-Thought*, asking the model to make its reasoning explicit in steps before answering, which reduces hallucinations when classifying spans with multiple operators. **N3** uses *Few-Shot*, embedding complete input–output examples that anchor the rigid JSON schema; since each RASE operator has a distinct nature, one prompt per operator was adopted (Applicability, Requirement, Selection, and Exception), each with the output schema, mapping heuristics based on common textual patterns (for example, "public use" → *property* "use", *comparation* "=", *target* "public"), and its own examples.

3.3. Evaluated LLMs

Six open-source models were evaluated, run locally through Ollama (Ollama, 2024), chosen for representing complementary profiles of size, alignment, language, and fine-tuning focus (Table 1). All share the decoder-only *Transformer* architecture and the paradigm of massive pre-training followed by instruction post-training, but differ in data, alignment, and scale. Except for Llama, evaluated in its official general-purpose multilingual version, all were run from Brazilian-Portuguese-adapted checkpoints published by third parties and available in the Ollama registry. Running the six models on the same hardware, with the same prompts and the same dataset, is what ensures the comparability of the results.

3.4. Dataset

The dataset was consolidated into a single JSON file with **79 annotated standards**, drawn from Brazilian AEC regulations, with emphasis on NBR 9050 (ABNT9050), adapted from the work of Mendonça and Manzione (2020). NBR 9050 was chosen for its mostly prescriptive profile, with numerical values directly verifiable in a BIM model and well-delimited exceptions. Each entry gathers the reference RASE decomposition across the three levels: the raw text of the standard (`text`); the list of atomic rules (`texts_n1`); for each rule, the four RASE operators (`operators_n2`, with the keys `requeriments`, `aplicability`, `selection`, and `exception`); and, for each operator, the classified span (`text_n2`) and the structured JSON properties (`properties_n3`). Absent operators preserve the structure with empty strings. The same file serves both as input to the generators and as reference for the validators, simplifying reproducibility.

3.5. Validation Metrics

The evaluation combines thirteen metrics across five families. The nine **similarity** metrics (introduced in Section 2.2) are computed pairwise between the predicted output and the reference, always on the normalized scale $[0, 1]$: lexical similarity (FuzzyWuzzy and TF-IDF), semantic (SBERT-PT `tgsc/sentence-transformer-ult5-pt-small`, Legal-BERTimbau, and `paraphrase-multilingual-mpnet-base-v2`), distance-based (WMD over FastText and NILC embeddings), and generation-oriented (BERTScore and ROUGE-L). To these are added four **classification** metrics (accuracy, precision, recall, and F1-score), which treat the evaluation as information extraction and make omissions and hallucinations explicit. An element is *positive* when filled and *negative* when empty (`""`); the matching criterion is semantic similarity above a threshold of 0.7 in N1 and N2, and exact equality (after normalizing case, accents, and punctuation) in N3. The 0.7 threshold balances rigor and tolerance to legitimate paraphrases; since it is applied equally to all models and experiments, it preserves the relative comparability among them.

Table 2
Per-metric averages across the five experiments, aggregated over the six models.

Metric	EN1	EN2	EN1N2	EN3	EN1N2N3
Lexical similarity					
FuzzyWuzzy	0.475	0.461	0.464	**0.847**	0.617
TF-IDF	0.636	0.312	0.417	0.597	0.514
Semantic similarity					
SBERT	**0.801**	0.490	0.594	**0.906**	**0.777**
BERTimbau	**0.816**	0.542	0.631	**0.836**	**0.752**
Multilingual	**0.850**	0.595	0.679	**0.873**	**0.793**
Semantic distance (WMD)					
WMD_ft	**0.716**	0.580	0.625	**0.846**	**0.757**
WMD_nilc	0.688	0.546	0.595	**0.842**	**0.742**
Generation-oriented metrics					
BERTScore	**0.866**	**0.753**	**0.790**	**0.880**	**0.843**
ROUGE-L	0.599	0.308	0.404	**0.765**	0.616
Classification metrics					
Accuracy	0.475	0.273	—	0.576	—
Precision	0.545	0.202	—	0.024	—
Recall	**0.774**	0.198	—	0.380	—
F1-score	0.631	0.197	—	0.043	—

3.6. Computational Environment and Implementation

All runs were carried out on a single workstation (Intel Core i9-13900K CPU, 32 GB of RAM, NVIDIA RTX 4080 GPU, Ubuntu 22.04, Python 3.11), with the models served by Ollama (`ollama serve`) and the metrics computed with `sentence-transformers`, `gensim`, `fuzzywuzzy`, and `scikit-learn`. The pipeline was implemented as a modular Python project, orchestrated by a main script exposing three paths (data generation, validation, and table generation). The code is organized into packages by responsibility: `config/` (model registry and connection to Ollama), `prompts/` (the prompts as text files, one per level and one per operator in N3), `generates/` and `validates/` (one module per step, with a symmetric structure), `utils/` (auxiliary routines for cleaning, pairing, and metric computation), and the annotated `dataset.json`. This separation keeps each level isolated and makes the pipeline reproducible: switching models means only changing a configuration entry, and each experiment can be regenerated or revalidated independently. The complete source code is available at `https://github.com/EikESousA/Rase-PE`.

4. Results and Discussion

This section presents the results of applying the six open-source LLMs to the 79 Brazilian standards in the dataset, across the five experiments: EN1 (isolated N1 segmentation), EN2 (RASE identification with reference N1), EN1N2 (chained N1→N2 pipeline), EN3 (JSON structuring in isolation, with reference N2), and EN1N2N3 (complete chained N1→N2→N3 pipeline). All figures come directly from the *metrics/validate_*.json* files, produced by the validation routine described in Section 3.5. For each experiment, the text reports the aggregate averages per metric, the detailed results per model, the execution times, and an integrated discussion.

Table-reading convention. In all numerical tables of this section, highlighting is done **per column**: the best value in each column appears in green and the worst in red. In the quality tables (similarity and classification), values in **bold** are those within the similarity threshold adopted in validation, that is, ≥ 0.7 (Section 3.5). In the execution-time tables, where *smaller is better*, green marks the fastest model and red the slowest, and bold does not apply.

4.1. Overview of the Experiments

Table 2 summarizes the per-metric averages across the five experiments, aggregated over the six models. Three patterns already stand out: embedding-based metrics (SBERT, BERTimbau, Multilingual, and BERTScore) keep consistently higher performance; the lexical ones (FuzzyWuzzy, TF-IDF, and ROUGE-L) lose ground as one moves from EN1 into EN1N2; and EN3 (isolated N3) rises to levels close to EN1 thanks to the restricted vocabulary of the JSON fields.

Table 3
Results per model in EN1 (similarity and classification).

	Lexical		Semantic			WMD		Generation	
Model	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0.473	0.561	**0.748**	**0.762**	**0.803**	0.694	0.664	**0.841**	0.540
Dolphin	0.641	**0.756**	**0.873**	**0.891**	**0.913**	**0.774**	**0.751**	**0.905**	**0.703**
Gemma	0.412	0.695	**0.842**	**0.850**	**0.885**	**0.726**	0.699	**0.883**	0.648
Llama	0.422	0.672	**0.835**	**0.844**	**0.873**	**0.711**	0.684	**0.871**	0.605
Mistral	0.534	**0.704**	**0.842**	**0.874**	**0.899**	**0.726**	0.699	**0.888**	0.651
Qwen	0.367	0.429	0.668	0.673	**0.726**	0.663	0.632	**0.808**	0.448

Classification metrics

Model	Accuracy	Precision	Recall	F1
Alpaca	0.419	0.512	0.699	0.591
Dolphin	0.676	**0.812**	**0.801**	**0.806**
Gemma	0.466	0.486	**0.917**	0.636
Llama	0.446	0.479	**0.865**	0.616
Mistral	0.610	0.670	**0.872**	**0.758**
Qwen	0.235	0.311	0.487	0.380

The semantic metrics (SBERT, BERTimbau, and Multilingual), together with BERTScore, better preserved aggregate value across the five experiments, as they are more robust to surface variation. The lexical ones and ROUGE-L suffer from the rewording produced by the LLMs. In N3, the textual-similarity metrics rise to much higher levels than in N2 (quasi-structured, short output), but the exact-match classification metrics required in EN3 expose systematic difficulties: macro precision falls below 4%. EN1N2N3 confirms the trend observed in EN1N2: the semantic metrics absorb error propagation well, while the lexical ones (TF-IDF and ROUGE-L) show a visible drop.

4.2. Results per Experiment

4.2.1. EN1: N1 Segmentation

In EN1, the comparison was between the N1 rules generated by the LLMs and the reference N1. The semantic metrics were well above the lexical ones (Multilingual 0.850, BERTimbau 0.816, and SBERT 0.801, against TF-IDF 0.636 and FuzzyWuzzy 0.475), indicating that segmentation tolerates lexical rewording when meaning is preserved. **Dolphin** dominated: it led in *all* nine similarity metrics and in three of the four classification metrics, with the best individual performance of the experiment. The detailed figures are in Table 3.

Figure 2 presents the visual comparison among the six models across the nine similarity metrics in EN1.

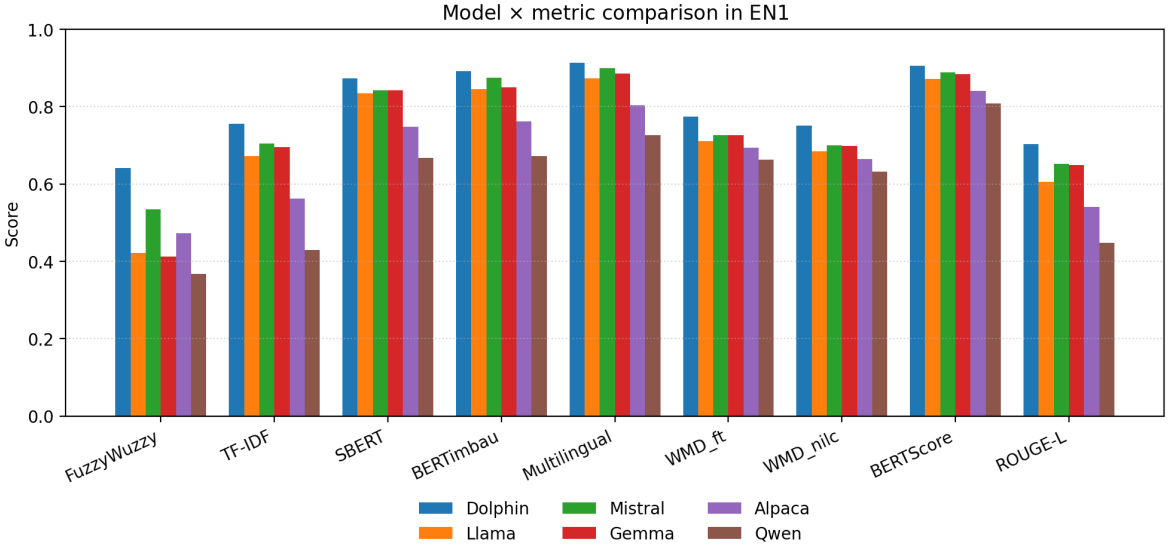

Figure 2: Model × metric comparison in EN1.

Table 4
Results per model in EN2 (similarity and per-operator macro classification).

	Lexical		Semantic			WMD		Generation	
Model	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0.334	0.168	0.386	0.418	0.484	0.522	0.486	**0.702**	0.172
Dolphin	0.511	0.423	0.557	0.621	0.665	0.620	0.589	**0.786**	0.412
Gemma	0.384	0.364	0.520	0.579	0.625	0.608	0.577	**0.772**	0.350
Llama	0.638	0.447	0.595	0.657	0.685	0.617	0.586	**0.796**	0.427
Mistral	0.563	0.385	0.575	0.607	0.665	0.607	0.575	**0.779**	0.377
Qwen	0.336	0.083	0.304	0.369	0.444	0.496	0.462	0.684	0.111

Classification metrics (macro per operator)

Model	Accuracy	Precision	Recall	F1
Alpaca	0.081	0.103	0.109	0.105
Dolphin	0.339	0.301	0.276	0.286
Gemma	0.182	0.252	0.300	0.265
Llama	0.439	0.237	0.250	0.243
Mistral	0.434	0.268	0.214	0.238
Qwen	0.160	0.051	0.039	0.044

4.2.2. EN2: RASE Operator Identification

EN2 measured the identification and structuring of the RASE operators from the reference N1. The averages dropped relative to EN1 (SBERT 0.490; Multilingual 0.595), reflecting the greater difficulty of structured formalization. The classification metrics fall even further (macro F1 0.197): the models have real difficulty distinguishing Selection and Exception. **Llama** leads in the semantic and lexical metrics; **Dolphin** takes the best macro F1 (0.286). The detailed figures are in Table 4.

Figure 3 consolidates the performance of the six models across the nine similarity metrics in EN2.

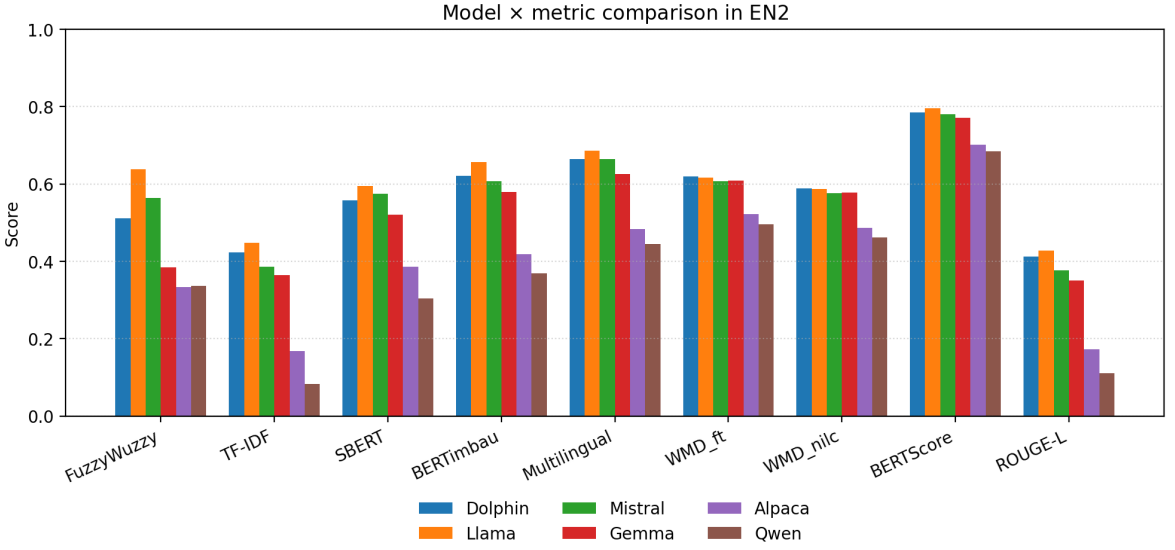

Figure 3: Model × metric comparison in EN2.

4.2.3. EN1N2: Chained Pipeline

EN1N2 tested the robustness of the fully automated pipeline: the N1 produced by the model in the first stage feeds the N2 of the same model. The results confirm the **error-propagation** hypothesis, but with a surprise. The averages drop relative to EN2 in some metrics and stay close or even improve in others, as in the semantic ones: Multilingual rises from 0.595 to 0.679; BERTimbau from 0.542 to 0.631; SBERT from 0.490 to 0.594. The reason is that the pipeline produces outputs closer to the original sentences and, therefore, more aligned with the reference text at the semantic level, even though it loses precision in operator assignment. Leadership is shared: **Dolphin** dominates

Table 5
Results per model in EN1N2 (similarity).

	Lexical		Semantic			WMD		Generation	
Model	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0.375	0.285	0.494	0.523	0.581	0.573	0.538	**0.742**	0.282
Dolphin	0.549	0.529	0.662	**0.711**	**0.749**	0.673	0.644	**0.826**	0.509
Gemma	0.391	0.458	0.615	0.656	**0.701**	0.646	0.617	**0.804**	0.436
Llama	0.570	0.521	0.677	**0.721**	**0.749**	0.649	0.620	**0.821**	0.488
Mistral	0.559	0.489	0.667	0.698	**0.740**	0.648	0.618	**0.817**	0.468
Qwen	0.342	0.219	0.448	0.478	0.551	0.562	0.530	**0.731**	0.240

TF-IDF, WMD_ft, WMD_nilc, BERTScore, and ROUGE-L; **Llama** leads SBERT and ties in Multilingual. The full figures are in Table 5.

Figure 4 presents the visual comparison in EN1N2, highlighting the alternating leadership between Dolphin and Llama across metrics.

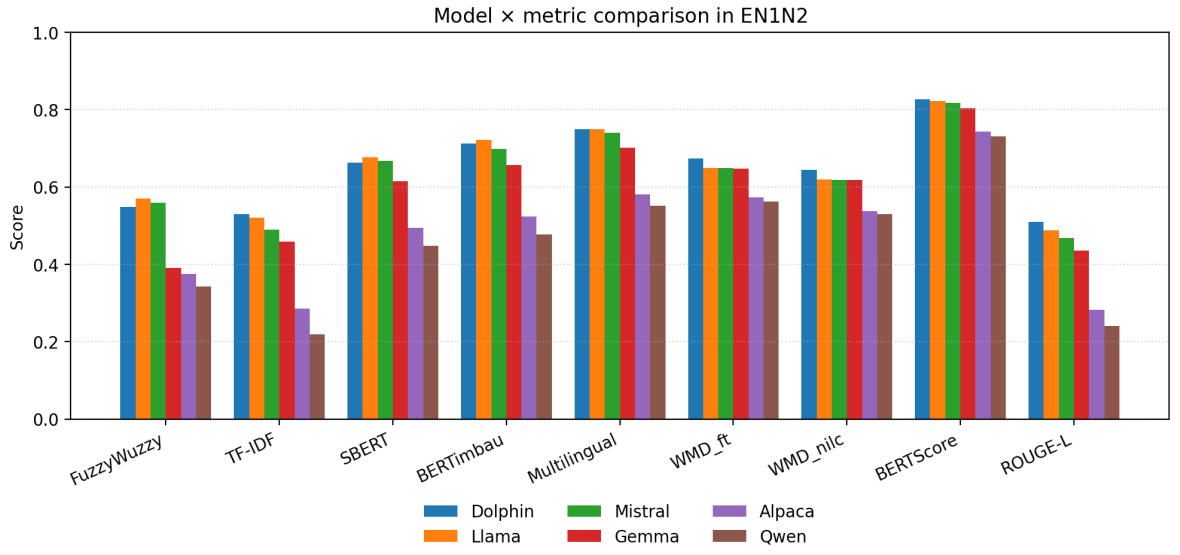

Figure 4: Model × metric comparison in EN1N2.

4.2.4. EN3: Semantic JSON Structuring with Reference N2

Level N3 (semantic JSON structuring) was evaluated in two complementary scenarios: EN3, in isolation, taking the reference N2 operators as input; and EN1N2N3, in the end-to-end chained pipeline, in which the generated N1 feeds the N2 of the same model, which in turn feeds the N3 of the same model. In both scenarios, the comparison is not between whole normative strings, but field by field (`type`, `object`, `property`, `comparation`, `target`, and `unit`), over the canonical serialization of the generated JSON against the annotated JSON.

Since the output is *quasi-structured* and made of short fields with restricted vocabulary, the textual-similarity metrics rise to much higher levels than in N1 and N2 (see Table 6). The exact-match classification metrics, by contrast, paint a much harsher picture: macro precision falls below 4%. This signals that the models preserve meaning but vary the vocabulary relative to the reference, for example `target = ‘‘public’’` instead of `target = ‘‘public use’’`. **Llama** dominated: it led in *all* nine similarity metrics, with SBERT 0.947 and Multilingual 0.928. Mistral, Dolphin, and Qwen are close in the semantic metrics, showing that the isolated N3 stage is the most homogeneous among the evaluated models.

The setup was identical to that of the previous experiments: the same six LLMs served through Ollama, the same per-operator few-shot prompts, the same dataset of 79 standards. Per-model outputs are stored in dedicated JSON files (`generate_n3_<model>.json`); the aggregate lives in a single metrics file (`validate_n3.json`). Exact per-field

Table 6
Results per model in EN3 (similarity and per-field macro classification).

	Lexical		Semantic			WMD		Generation	
Model	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	**0.729**	0.395	**0.846**	**0.745**	**0.798**	**0.756**	**0.751**	**0.815**	0.624
Dolphin	**0.860**	0.694	**0.912**	**0.848**	**0.887**	**0.842**	**0.839**	**0.901**	**0.799**
Gemma	**0.751**	0.480	**0.858**	**0.755**	**0.823**	**0.762**	**0.757**	**0.810**	0.652
Llama	**0.892**	**0.736**	**0.947**	**0.897**	**0.928**	**0.915**	**0.912**	**0.925**	**0.871**
Mistral	**0.877**	0.689	**0.938**	**0.891**	**0.914**	**0.903**	**0.900**	**0.922**	**0.842**
Qwen	**0.872**	0.590	**0.937**	**0.877**	**0.889**	**0.898**	**0.895**	**0.905**	**0.804**

Classification metrics (macro per field)

Model	Accuracy	Precision	Recall	F1
Alpaca	0.283	0.012	0.434	0.024
Dolphin	**0.705**	0.034	0.447	0.062
Gemma	0.292	0.017	0.603	0.033
Llama	**0.725**	0.038	0.361	0.067
Mistral	**0.760**	0.037	0.371	0.066
Qwen	0.692	0.004	0.063	0.008

Table 7
Results per model in EN1N2N3 (similarity over the structured JSON).

	Lexical		Semantic			WMD		Generation	
Model	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0.525	0.345	0.687	0.645	**0.701**	0.676	0.658	**0.782**	0.471
Dolphin	0.685	0.600	**0.811**	**0.794**	**0.832**	**0.776**	**0.762**	**0.871**	0.683
Gemma	0.536	0.419	**0.747**	**0.710**	**0.767**	**0.712**	0.697	**0.807**	0.553
Llama	**0.701**	0.630	**0.834**	**0.823**	**0.853**	**0.804**	**0.790**	**0.882**	**0.711**
Mistral	0.693	0.600	**0.830**	**0.814**	**0.845**	**0.801**	**0.788**	**0.879**	0.691
Qwen	0.561	0.490	**0.750**	**0.724**	**0.762**	**0.773**	**0.759**	**0.838**	0.590

matching is harsher than the similarity threshold adopted in N1 and N2, which drives down the observed precision, especially in the models with weaker schema adherence (Alpaca and Qwen).

4.2.5. EN1N2N3: Complete Chained Pipeline

EN1N2N3 is the end-to-end automated pipeline. This is the scenario that replicates real production use: the sole input is the raw text of the standard, and the final output is the per-operator structured JSON. Since each stage accumulates transformations, it is also the experiment most subject to **error propagation**: the averages drop relative to isolated EN3 in all metrics (SBERT 0.777 *vs.* 0.906; Multilingual 0.793 *vs.* 0.873; BERTScore 0.843 *vs.* 0.880). Even so, the absolute levels remain higher than those observed in EN2 and EN1N2, benefiting from the restricted JSON vocabulary. Table 7 reports the results per model.

Each model's output is persisted in its own JSON file (*generate_n1n2n3_<model>.json*), and the aggregate metrics live in a single file (*validate_n1n2n3.json*). **Llama** keeps the lead it earned in isolated EN3, now across all nine similarity metrics; **Mistral** comes immediately behind, with marginal differences (Multilingual 0.845 *vs.* 0.853). In line with the pattern observed in EN1N2, Llama, Mistral, and Dolphin retain their relative advantage in the full chain as well, whereas Alpaca and Gemma accumulate a drop across all metrics and deliver the worst overall performance, showing that failures in N1/N2 contaminate the final N3 structuring even in the semantic metrics.

4.3. Results per Model

Table 8 reports the overall average per model, aggregating the nine textual-similarity metrics over the five experiments (EN1, EN2, EN1N2, EN3, and EN1N2N3). The final ranking departs from expectation: the leader is **Llama**, sustained by its absolute lead in EN3 and EN1N2N3 (both N3 stages) and by the best performance in EN2. **Dolphin** comes second: it dominates EN1 by a wide margin and keeps competitive performance in the following stages. Mistral comes third, followed by Gemma, Qwen, and Alpaca.

The per-model observations follow.

Llama. Leads the overall average (0.730) and dominates the N3-level experiments: in isolated EN3 it was ahead in *all* nine metrics (SBERT 0.947; Multilingual 0.928; BERTScore 0.925); in EN1N2N3, it repeated the lead across the nine metrics (SBERT 0.834; Multilingual 0.853; BERTScore 0.882). It was also the best in EN2 in seven metrics: FuzzyWuzzy (0.638), TF-IDF (0.447), SBERT (0.595), BERTimbau (0.657), Multilingual (0.685), BERTScore (0.796), and ROUGE-L (0.427), in addition to the highest macro classification accuracy in the same

Table 8
Overall average per model (nine similarity metrics × five experiments).

Model	EN1	EN2	EN1N2	EN3	EN1N2N3	Overall Avg.
Llama	**0.724**	0.605	0.646	**0.892**	**0.781**	**0.730**
Dolphin	**0.801**	0.576	0.650	**0.843**	**0.757**	**0.725**
Mistral	**0.757**	0.570	0.634	**0.875**	**0.771**	**0.721**
Gemma	**0.738**	0.531	0.591	**0.739**	0.661	0.652
Qwen	0.602	0.366	0.456	**0.852**	0.694	0.594
Alpaca	0.676	0.408	0.488	**0.718**	0.610	0.580

experiment (0.439). In EN1N2, it led in FuzzyWuzzy (0.570), SBERT (0.677), and BERTimbau (0.721), tying with Dolphin in Multilingual (0.749). It is the choice when one can tolerate the computational cost.

Dolphin. It was the best segmenter in EN1, with absolute dominance in the nine metrics (BERTimbau 0.891; Multilingual 0.913; SBERT 0.873; BERTScore 0.905) and the best macro classification F1 in the experiment (0.806). In EN2, it led in WMD_ft (0.620), WMD_nilc (0.589), and macro F1 (0.286). In EN1N2, it was ahead in TF-IDF (0.529), WMD_ft (0.673), WMD_nilc (0.644), BERTScore (0.826), and ROUGE-L (0.509). In EN3 and EN1N2N3, it keeps competitive performance (SBERT 0.912 and 0.811, respectively), though behind Llama and Mistral. Added to this is its low computational cost (Section 4.5), which yields the best cost-benefit ratio observed.

Mistral. Consistent performance across all scenarios, and the most regular of the batch: third in the overall average (0.721), less than one percentage point behind the leader. It was close to the leaders in EN1 (Multilingual 0.899; BERTimbau 0.874) and in EN1N2 (Multilingual 0.740; BERTimbau 0.698), delivered the second-best macro accuracy in EN2 (0.434), and stays right next to Llama in EN3 (SBERT 0.938) and in EN1N2N3 (SBERT 0.830).

Gemma. Appears as the leader in macro classification recall in both EN1 (0.917) and EN2 (0.300), that is, good coverage with lower precision. Its semantic averages sit at an intermediate level.

Alpaca. Weak performance across all experiments, with a sharp drop in EN2 (SBERT 0.386; macro F1 0.105).

Qwen. Worst overall performance. It collapsed in EN2 (TF-IDF 0.083; macro F1 0.044) and had the highest latency in the chained pipeline (see Section 4.5). It is also weighed down by the model's tendency to generate outputs in simplified Chinese, outside the expected language.

4.4. Analysis per Metric

Lexical metrics. FuzzyWuzzy and TF-IDF drop clearly from EN1 to EN2 (FuzzyWuzzy: 0.475 → 0.461; TF-IDF: 0.636 → 0.312). Since they rely on lexical overlap, these metrics penalize the syntactic rewordings and synonymies generated by the LLMs. In EN3 and EN1N2N3, they rise again (FuzzyWuzzy: 0.847 and 0.617; TF-IDF: 0.597 and 0.514), benefiting from the short, restricted JSON vocabulary.

Semantic metrics. SBERT, BERTimbau, and Multilingual keep high and stable averages. BERTimbau, trained on Portuguese legal text, sits slightly above SBERT in almost every model and experiment (EN1: 0.816 *vs.* 0.801; EN1N2: 0.631 *vs.* 0.594), practical evidence of the gain from using domain embeddings to validate normative transformations. In EN3, they reach the highest values of the batch (SBERT 0.906; Multilingual 0.873), because the JSON fields contain short phrases with low lexical variation.

Generation-oriented metrics. BERTScore had the highest absolute average in EN2 (0.753) and in EN1N2 (0.790), thanks to the optimal alignment between tokens even under strong rewording. In EN3 and EN1N2N3, it stays high (0.880 and 0.843). ROUGE-L, in turn, behaved like a lexical metric, with a sharp drop in EN2 (0.308) and recovery in EN3 (0.765).

Distance metrics (WMD). Intermediate performance, between lexical and semantic. The values were highest in EN1 (WMD_ft 0.716; WMD_nilc 0.688) and in EN3 (WMD_ft 0.846; WMD_nilc 0.842), reflecting the greater lexical closeness between outputs and the reference text/JSON in those two scenarios.

Classification metrics. In EN1, macro F1 reaches 0.806 (Dolphin), indicating that most atomic sentences are segmented correctly. In EN2, F1 collapses to 0.286 in the best case (Dolphin), with a breakdown in Selection and Exception: F1 close to zero for all models. These two operators are especially hard to identify even for the leaders. In EN3, the low exact-match precision (macro precision below 4% for all models) shows that token-by-token matching is too severe for a field whose semantics resists legitimate lexical variation.

Table 9
Total generation time per model in N1.

Model	Time (s)	Time (h:mm:ss)
Alpaca	104.55	0:01:44
Dolphin	62.65	0:01:02
Gemma	133.04	0:02:13
Llama	5,676.97	1:34:36
Mistral	70.04	0:01:10
Qwen	309.59	0:05:09

Table 10
Total generation time per model in N2.

Model	Time (s)	Time (h:mm:ss)
Alpaca	249.10	0:04:09
Dolphin	124.69	0:02:04
Gemma	244.19	0:04:04
Llama	6,895.71	1:54:55
Mistral	123.63	0:02:03
Qwen	6,515.09	1:48:35

Table 11
Total generation time per model in N1N2.

Model	Time (s)	Time (h:mm:ss)
Alpaca	330.84	0:05:30
Dolphin	104.28	0:01:44
Gemma	427.22	0:07:07
Llama	9,450.14	2:37:30
Mistral	138.39	0:02:18
Qwen	10,093.84	2:48:13

Table 12
Total generation time per model in N3 (isolated EN3).

Model	Time (s)	Time (h:mm:ss)
Alpaca	2,104.92	0:35:05
Dolphin	584.96	0:09:45
Gemma	5,982.38	1:39:42
Llama	800.78	0:13:21
Mistral	627.68	0:10:28
Qwen	847.41	0:14:07

4.5. Execution Time

The total generation time was measured per model and per experiment. Table 9 reports the times in N1; Table 10, in N2; Table 11, in the chained N1N2 pipeline; Table 12 reports the EN3 times (isolated N3, with reference N2 as input); and Table 13, those of the complete EN1N2N3 pipeline.

The times expose a clear *trade-off* between quality and computational cost. In EN1 and EN1N2, Llama and Qwen are the slowest: respectively 1h34min and 5min in N1; 2h37min and 2h48min in N1N2. In EN3 and EN1N2N3, the picture inverts: **Gemma** becomes the most expensive (1h39min in N3; 1h46min in N1N2N3), while Llama, Mistral, and Qwen sit around 10 to 14 minutes. Dolphin combines high quality with times on the order of 1 to 13 minutes in all experiments (62 s in N1; 1m44s in N1N2; 9m45s in N3; 12m38s in N1N2N3), and remains the best cost-benefit ratio observed in the batch.

Table 13
Total generation time per model in N1N2N3 (complete pipeline).

Model	Time (s)	Time (h:mm:ss)
Alpaca	1,395.35	0:23:15
Dolphin	757.65	0:12:38
Gemma	6,364.48	1:46:04
Llama	808.62	0:13:29
Mistral	637.88	0:10:38
Qwen	808.72	0:13:29

4.6. Integrated Discussion

Reading the experiments and the thirteen metrics together allows five observations to be consolidated.

1. Semantic embeddings and BERTScore are the metrics best suited to this task. SBERT, BERTimbau, Multilingual, and BERTScore capture equivalence even under heavy lexical rewording, and are the most stable metrics across the experiments. BERTimbau, trained on a PT-BR legal corpus, sits slightly above SBERT, evidence of the gain from domain embeddings.

2. The chained pipeline does not degrade all metrics uniformly. The drop expected from error propagation appears in the lexical metrics (FuzzyWuzzy, TF-IDF, and ROUGE-L). In the semantic ones, however, EN1N2 even *surpasses* isolated EN2 in the aggregate averages (Multilingual 0.679 *vs.* 0.595): the N1 produced by the model itself tends to generate sentences close to the originals, which favors semantic matching, even when N2 errs in operator assignment. The classification metrics confirm, nonetheless, that this gain does not translate into better RASE classification: the pipeline still depends on atomized and correctly labeled output for automatic checking to make sense.

3. There are complementary profiles among the leading models. Llama leads overall quality (0.730), with absolute dominance over level N3 (the nine metrics in EN3 and in EN1N2N3) and also over EN2; it is the choice when one can tolerate the computational cost. Dolphin (0.725) delivers the best cost-benefit: it dominates EN1, is ahead in the EN1N2 distance metrics, and has the shortest generation time among the leaders. Mistral (0.721) is third and the most consistent: it never leads by a wide margin, but is never far behind.

4. Qwen and Alpaca are not recommended. Both have consistently inferior performance across all metrics and experiments. Qwen also records the highest latency in the chained pipeline (2h48min) and tends to generate outputs outside the expected language.

5. The three-level decomposition was decisive. Separating segmentation (N1), RASE classification (N2), and JSON structuring (N3) made valid outputs possible without using *Fine-Tuning* or *Retrieval-Augmented Generation*; that is, Prompt Engineering alone suffices to reach semantic similarity near 0.90 in the best scenarios (Dolphin in EN1, with BERTimbau 0.891; Llama in isolated N3, with SBERT 0.947). The remaining difficulty is concentrated in the correct classification of the Selection and Exception operators in N2, the hardest classes for all evaluated models.

5. Conclusion

This work investigated the application of six open-source LLMs (Llama, Dolphin, Gemma, Mistral, Alpaca, and Qwen) to the automatic conversion of Brazilian AEC technical standards to the RASE format, using **Prompt Engineering only**. The task was decomposed into three successive normalization levels (N1, N2, and N3) and evaluated across five experiments (EN1, EN2, EN1N2, EN3, and EN1N2N3), over a dataset of 79 annotated standards. The evaluation combined thirteen metrics: nine of textual similarity (lexical, semantic, distance-based, and generation-oriented) and four of classification, derived from the confusion matrix.

5.1. Summary of the Results

The three-level decomposition enabled the conversion of AEC standards into RASE representations without resorting to *Fine-Tuning* or *Retrieval-Augmented Generation*. Prompt Engineering alone sufficed to reach semantic similarity near 0.90 in the best scenarios (Dolphin in EN1, with BERTimbau 0.891 and Multilingual 0.913; Llama in isolated EN3, with SBERT 0.947; and again Llama in EN1N2N3, with SBERT 0.834 even in the full chain). The embedding-based metrics (SBERT, BERTimbau, Multilingual) and BERTScore were the most suitable for validating normative transformations, because they tolerate rewordings that preserve meaning. BERTimbau kept a consistent gain over SBERT, practical evidence of the value of PT-BR legal-domain embeddings.

In the aggregate of the nine similarity metrics over the five experiments, **Llama** closed with the best overall average (0.730), driven by its absolute lead in EN3 and EN1N2N3 (the two N3-level stages) and in EN2. **Dolphin** came second (0.725), with the best cost-benefit ratio: about ninety times faster than Llama in the chained pipeline, and the absolute leader in EN1. Mistral closed third (0.721), the most consistent of the batch. Qwen and Alpaca showed unsatisfactory performance in the stages most sensitive to semantic failures (EN2 and EN1N2), but recovered in EN3, benefiting from the restricted JSON vocabulary. The classification metrics confirmed an important finding: the Selection and Exception operators are the hardest classes in N2 for all models, with F1 close to zero; in EN3, exact per-field matching drives macro precision below 4%, pointing to the need for more flexible classification criteria at level N3.

5.2. Contributions

The research delivers five central contributions:

- The operational definition of three normalization levels over the RASE methodology: **N1** (atomic segmentation), **N2** (RASE operator identification), and **N3** (semantic JSON structuring), with prompts dedicated to each level and each operator.
- A systematic comparison of six open-source LLMs served locally through Ollama, on the task of converting Brazilian AEC standards, under an identical experimental protocol (same dataset, same prompts, same hardware).
- A reproducible set of specialized prompts per level and per operator, organized into dedicated text files, ready to be reused and extended in future research.
- An annotated dataset of 79 Brazilian standards, with reference at every level (N1, N2, and N3) and per RASE operator, enabling replicable evaluations.
- A multi-metric evaluation with thirteen metrics (nine of textual similarity and four of classification), delivering a multidimensional portrait of each model's performance, with per-instance detail in `metrics/details/` to support reanalyses.

5.3. Limitations

The research has limitations that bound the reach of its conclusions:

- The dataset is small (79 standards) and concentrated in Brazilian accessibility standards (NBR 9050 and similar), so the results may change in other normative domains.
- The classification metrics depend on the adopted hit criterion: a threshold over semantic similarity in N1 and N2 (partly arbitrary) and exact matching in N3 (too rigid for fields with legitimate lexical variation). More flexible criteria in N3 and multiple thresholds in N1 and N2 remain a line for deeper study.
- The classification of Selection and Exception in N2 obtained F1 close to zero for all models, signaling a structural limitation of the current N2 prompt formulation: none of the six LLMs managed to overcome it.
- There was no practical application in BIM *software*. End-to-end validation in real projects is still open; the complete automatic compliance-checking cycle needs to be closed.
- All experiments ran on a single workstation, without variance analysis across runs, without temperature variation, and without seed variation.

5.4. Future Work

Some directions for future research considered promising:

- **BIM application**: integrate the results into Revit or Dynamo and close the automatic compliance-checking cycle in real projects.
- **N2 prompt dedicated to Selection and Exception**: redesign the N2 prompt or introduce a specific stage for these two operators. Both collapsed to F1 close to zero in the six models, which makes this the point of greatest leverage.

- **Fine-Tuning and RAG**: explore these techniques as natural evolutions of the prompt-only approach, measuring the gain in quality and computational cost, especially for the hard operators cited above.
- **Dataset expansion**: include a wider spectrum of NBRs (structural, hydraulic, electrical) and extend to international standards.
- **Human evaluation**: complement the automatic metrics with evaluation by AEC specialists, able to capture semantic nuances that escape the embeddings.
- **Variance study**: run multiple executions per model, varying temperature and seed, to quantify output stability.
- **Alternative classification criteria in N3**: replace exact matching with normalized matching (lowercase, stopword removal, synonym comparison) and revisit Accuracy, Precision, Recall, and F1 in N3.
- **Pipeline optimization**: study strategies to mitigate error propagation in EN1N2 and EN1N2N3: iterative rewriting, *self-consistency*, or routing hard operators to the most suitable model (a Dolphin/Llama ensemble per level).

Overall, the work shows that open-source LLMs, combined with a careful decomposition of the task and specialized prompts, are a viable and accessible alternative for automating the conversion of AEC technical standards into computable representations, paving the way for automatic compliance checking in BIM-based projects.

CRediT authorship contribution statement

Eike Natan Sousa Brito: Conceptualization, Methodology, Software, Writing - original draft. **André Carvalho:** Supervision, Writing - review & editing. **Marco Antônio Brasiel Sampaio:** Supervision, Writing - review & editing.

References

ABNT9050, 2020. NBR 9050: Acessibilidade a edificações, mobiliário, espaços e equipamentos urbanos. Associação Brasileira de Normas Técnicas. Rio de Janeiro.

AI@Meta, 2024. The Llama 3 herd of models. Model Card. URL: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.

Alawadhi, M., Yan, W., 2023. Deep learning from parametrically generated virtual buildings for real-world object recognition. arXiv preprint arXiv:2302.05283 URL: https://arxiv.org/abs/2302.05283.

Beach, T.H., Rezgui, Y., Li, H., Kasim, T., 2015. A rule-based semantic approach for automated regulatory compliance in the construction sector. Expert Systems with Applications 42, 5219–5231. doi:10.1016/j.eswa.2015.02.029.

Bojanowski, P., Grave, E., Joulin, A., Mikolov, T., 2017. Enriching word vectors with subword information. Transactions of the Association for Computational Linguistics 5, 135–146.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., et al., 2020. Language models are few-shot learners. URL: https://arxiv.org/abs/2005.14165, arXiv:2005.14165.

Davenport, T.H., Patil, D.J., 2012. Data scientist: The sexiest job of the 21st century. Harvard Business Review 90, 70–76. URL: https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century.

Dimyadi, J., Amor, R., 2013. Automated building code compliance checking – where is it at?, in: Proceedings of the 19th International CIB World Building Congress, Brisbane. pp. 172–185.

Eastman, C., Lee, J.M., Jeong, Y.S., Lee, J.K., 2009. Automatic rule-based checking of building designs. Automation in Construction 18, 1011–1033. doi:10.1016/j.autcon.2009.07.002.

Eastman, C., Teicholz, P., Sacks, R., 2011. BIM Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers and Contractors. 2 ed., Wiley, New Jersey.

Fuchs, S., Witbrock, M., Dimyadi, J., Amor, R., 2024. Using large language models for the interpretation of building regulations. arXiv preprint arXiv:2407.21060 URL: https://arxiv.org/abs/2407.21060.

Hjelseth, E., 2015. BIM-based model checking (BMC), in: Building Information Modeling: Applications and Practices. ASCE, pp. 33–61. doi:10.1061/9780784413982.ch02.

Hjelseth, E., Nisbet, N., 2011. Capturing normative constraints by use of the semantic mark-up RASE methodology, in: Proceedings of the CIB W78-W102 International Conference, pp. 1–10. URL: https://api.semanticscholar.org/CorpusID:62759547.

ISO19650, 2018. ISO 19650-1: Organization and digitization of information about buildings and civil engineering works, including building information modelling (BIM) – Information management using building information modelling – Part 1: Concepts and principles. International Organization for Standardization. Geneva.

Jurafsky, D., Martin, J.H., 2009. Speech and Language Processing. 2 ed., Prentice-Hall.

Kusner, M.J., Sun, Y., Kolkin, N.I., Weinberger, K.Q., 2015. From word embeddings to document distances, in: Proceedings of the 32nd International Conference on Machine Learning (ICML), pp. 957–966.

Lin, C.Y., 2004. ROUGE: A package for automatic evaluation of summaries, in: Text Summarization Branches Out: Proceedings of the ACL-04 Workshop, Association for Computational Linguistics. pp. 74–81.

Lin, W.Y., 2023. Prototyping a chatbot for site managers using building information modeling (BIM) and natural language understanding (NLU) techniques. Sensors 23, 2942. doi:10.3390/s23062942.

Liu, Y., Zhang, X., Li, H., 2022. BIM, machine learning and design rules to improve the assembly process. Sustainability 14, 288. URL: https://www.mdpi.com/2071-1050/14/1/288.

Manning, C.D., Raghavan, P., Schütze, H., 2008. Introduction to Information Retrieval. Cambridge University Press, Cambridge, UK.

Mendonça, E.A.d., Manzione, L., 2020. Conversão de regras de acessibilidade pela metodologia RASE para verificação automática em modelo BIM.

Ollama, 2024. Ollama: Get up and running with large language models locally. https://ollama.com.

OpenAI, 2024. GPT-4 technical report. URL: https://arxiv.org/abs/2303.08774, arXiv:2303.08774.

Rane, N., Choudhary, S., Rane, J., 2023. Integrating building information modelling (BIM) with ChatGPT, Bard, and similar generative artificial intelligence in the architecture, engineering, and construction industry: applications, a novel framework, challenges, and future scope. SSRN Electronic Journal doi:10.2139/ssrn.4645601.

Reimers, N., Gurevych, I., 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics. pp. 3982–3992. URL: https://arxiv.org/abs/1908.10084.

Saka, A., Taiwo, R., Saka, N., Oluleye, B., Dauda, J., Akanbi, L., 2024. Integrated BIM and machine learning system for circularity prediction of construction demolition waste. arXiv preprint arXiv:2407.14847 URL: https://arxiv.org/abs/2407.14847.

Sammour, F., Xu, J., Wang, X., Hu, M., Zhang, Z., 2024. Responsible AI in construction safety: Systematic evaluation of large language models and prompt engineering. URL: https://arxiv.org/abs/2411.08320, arXiv:2411.08320.

Samsami, R., 2024. Optimizing the utilization of generative artificial intelligence (AI) in the AEC industry: ChatGPT prompt engineering and design. CivilEng 5, 971–1010. doi:10.3390/civileng5040049.

Santos, K.T., Sampaio, M.A.B., 2021. Aplicação da metodologia RASE na norma de acessibilidade associada ao BIM. Simpósio Brasileiro de Gestão e Economia da Construção 12, 1–8. doi:10.46421/sibragec.v12i00.500.

Solihin, W., Eastman, C., 2015. Classification of rules for automated BIM rule checking development. Automation in Construction 53, 69–82. doi:10.1016/j.autcon.2015.03.003.

Souza, F., Nogueira, R., Lotufo, R., 2020. BERTimbau: Pretrained BERT models for Brazilian Portuguese, in: 9th Brazilian Conference on Intelligent Systems (BRACIS), Springer. pp. 403–417.

Tan, P.N., Steinbach, M., Kumar, V., 2006. Introduction to Data Mining. 1 ed., Pearson Education, Boston.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G., 2023. LLaMA: Open and efficient foundation language models. URL: https://arxiv.org/abs/2302.13971, arXiv:2302.13971.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 5998–6008. URL: https://arxiv.org/abs/1706.03762.

Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y., 2020. BERTScore: Evaluating text generation with BERT. URL: https://arxiv.org/abs/1904.09675, arXiv:1904.09675. 8th International Conference on Learning Representations (ICLR).

Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Nie, J.Y., Wen, J.R., 2023. A survey of large language models. arXiv preprint arXiv:2303.18223 URL: https://arxiv.org/abs/2303.18223.